24/5/2017

EE 241 DIGITAL CIRCUITS
FINAL EXAM SOLUTION MANUAL

Name:

Q1) (20 points) Use a button and LED on the Basys3 board such that when the button is pressed once, the LED turns on. When it is pressed second time, the LED turns off. Use a suitable flip-flop description for this operation in Verilog.

Solution:

```
module toggle_LED(led, button, clk);

input button;
input clk;
output reg led;

initial
led=0;

always @(posedge clk)
if (button)
led=~led;

endmodule


`timescale 1ns / 1ps
module toggle_LED_tb;

reg button;
reg clk;
wire led;

toggle_LED UUT(led, button, clk);

initial begin
#1 clk=0;
forever
#1 clk=~clk;
end

initial
begin
#1  button = 0;
#1  button = 1;
```

```
#1  button = 0;
#10 button = 0;

#1  button = 1;
#1  button = 0;
#10 button = 0;
#1  button = 1;
#1  button = 0;

end
endmodule
```

Q2) (30 points) You are asked to construct a Verilog module which will count the clock pulses between two successive low to high transitions of an input signal. This is in fact the working principle of the capture mode in timers in a microcontroller. This way, the duration of an operation can be calculated in terms of clock cycles.

Let's use the clock signal, *clk*, available in the Basys3 board. The input signal to be observed is called *in1* in your module. The maximum count value will be 0xFFFF. If the second low to high transition comes after this value, then an overflow signal, called *ovr*, should be set to indicate that the count value exceeded its limit.

Solution

```
module capture(count, ovr, in1, clk);

input in1;
input clk;
output reg [3:0] count;
output reg ovr;

reg cond;

initial
cond=0;

always @(posedge clk)
begin
if (in1)
cond=~cond;

if (in1 & cond)
count=0;

if(cond)
{ovr,count} <= count+1'b1;
end;

endmodule


`timescale 1ns / 1ps
module capture_tb;

reg in1;
reg clk;
wire [3:0] count;
wire ovr;

capture UUT(count, ovr, in1, clk);
```

```
initial begin
#1 clk=0;
forever
#1 clk=~clk;
end

initial
begin
#1 in1 = 1;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 1;
#2 in1 = 0;
#2 in1 = 1;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 0;
#2 in1 = 1;
#2 in1 = 0;

end
endmodule
```

Q3) (20 points) You are asked to form a Verilog description (using behavioral modeling and if statements) to design a complete arithmetic logic unit (ALU) module in a microcontroller. The ALU has the following properties.

The unit can only perform addition and subtraction operations on two eight-bit numbers. Only addition operation can perform both addition and subtraction operations. Therefore, remember the two's complement representation.
The arithmetic unit will have its inputs as follows.
num1: first number to be processed which is 8 bits.
num2: second number to be processed which is 8 bits.
op_type: operation type represented in one bit. For addition and subtraction, *op_type* will be 0 and 1 respectively.
clk: clock signal
The arithmetic unit will have one output as follows.
res: the result of the operation which is 8 bits.

We will also add four flags representing the result of the operation. These flags are as follows.
N: will be equal to logic level 1 when the result is negative. Otherwise, it will be zero.
C: will be equal to logic level 1 when a carry bit is generated in addition.
V: will be equal to logic level 1 when an overflow occurs in addition or subtraction. Otherwise, it will be zero.
Z: will be equal to logic level 1 when the result is zero. Otherwise, it will be zero.

Note: This is the question asked in Midterm. Do not provide the answers given in the midterm solution manual directly since we used data flow modeling there.

Solution:

```
module ALU(res,N,C,V,Z,num1,num2,op_type,clk);

output reg [7:0] res;
output reg N,C,V,Z;
input [7:0] num1, num2;
input op_type, clk;

reg [8:0] add_res, sub_res;

initial
begin
N=0;
C=0;
V=0;
Z=0;
add_res=9'b000000000;
sub_res=9'b000000000;
end
```

```
always @(posedge clk)
begin

//Addition and Subtraction
add_res=num1+num2;
sub_res=num1+~num2+1'b1;

//Overflow, V
if(op_type)
{V,res} = sub_res;
else
{V,res} = add_res;

//Negative, N
if (sub_res[8]&op_type)     N=1;
//Carry, C
if (add_res[8]&~op_type)    C=1;
//Zero, Z
if (res==8'h00)           Z=1;

end

endmodule

module ALU_tb;

reg [7:0] num1,num2;
reg op_type,clk;
wire [7:0] res;
wire N,C,V,Z;

ALU UUT(res,N,C,V,Z,num1,num2,op_type,clk);

initial begin
#1 clk=0;
forever
#1 clk=~clk;
end

initial begin
num1 = 8'hFF;
num2 = 8'h03;
op_type=1'b0;

#10;
end
endmodule
```

Q4) (30 points) An external digital system provides characters (in ASCII form) every clock cycle. We are only interested in counting the occurrence of characters ‘a’, ‘b’, ’c’, and ‘d’. Write a Verilog module which gives the total number of each character of interest as output. Input to the module is the character in ASCII form and the clock cycle.

Solution

```
module histogram(ac, bc, cc, dc, inchar, clk);

input [7:0] inchar;
input clk;
output reg [7:0] ac, bc, cc, dc;

initial
begin
ac =0;
bc =0;
cc =0;
dc =0;
end

always @(posedge clk)
case(inchar)
  8'h61 : ac = ac+1;
  8'h62 : bc = bc+1;
  8'h63 : cc = cc+1;
  8'h64 : dc = dc+1;
endcase

endmodule
```

```
`timescale 1ns / 1ps
module histogram_tb;

reg [7:0] inchar;
reg clk;
wire [7:0] ac, bc, cc, dc;

histogram UUT(ac, bc, cc, dc, inchar, clk);

initial begin
#1 clk=0;
forever
#1 clk=~clk;
end
```

```
initial
begin
#1;
#2 inchar = 8'h61;
#2 inchar = 8'h62;
#2 inchar = 8'h63;
#2 inchar = 8'h64;
#2 inchar = 8'h64;
#2 inchar = 8'h64;
#2 inchar = 8'h61;
#2 inchar = 8'h00;
end
endmodule
```